

JAVA – MODULE 5

String Handling (Detailed Theory + Simple Examples for Full Marks)

1. Introduction to String Class

In Java, a **String** is a sequence of characters represented as an object of the `String` class. The `String` class is part of the `java.lang` package, which means it is available to all Java programs by default.

Java treats strings as **objects**, not primitive data types. This allows Java to provide many built-in methods for string manipulation such as comparison, searching, concatenation, and case conversion.

Key Characteristics

- Strings are objects of `String` class
- Stored as immutable objects
- Rich set of built-in methods

Simple Example

```
String s = "Hello";
```

2. Immutability of Strings

Strings in Java are **immutable**, meaning once a `String` object is created, its contents cannot be changed.

If any operation appears to modify a string, Java actually creates a **new String object**, leaving the original unchanged.

Why Immutability?

- Security
- Thread safety
- Performance optimization (String pool)

Example

```
String s = "Java";  
s = s + " Programming"; // new object created
```

3. Mutable Strings: StringBuffer and StringBuilder

If modification is required, Java provides: - `StringBuffer` (thread-safe) - `StringBuilder` (not thread-safe, faster)

Both allow modification after creation.

4. String Constructors

Empty String

```
String s = new String();
```

From Character Array

```
char[] ch = {'a', 'b', 'c'};  
String s = new String(ch);
```

From Subrange of Character Array

```
char[] ch = {'a', 'b', 'c', 'd', 'e'};  
String s = new String(ch, 2, 3); // cde
```

From Another String

```
String s1 = "Java";  
String s2 = new String(s1);
```

From Byte Array

```
byte[] b = {65, 66, 67};  
String s = new String(b); // ABC
```

5. String Length

The length of a string refers to the number of characters it contains.

```
String s = "Hello";  
System.out.println(s.length()); // 5
```

6. Special String Operations

Java provides special support for string operations: - String literals - Concatenation using `+` - Automatic type conversion

7. String Literals

Java automatically creates String objects for literals.

```
String s = "Java";  
System.out.println("Hello".length());
```

8. String Concatenation

The `+` operator concatenates strings.

```
String s = "Java" + " Programming";
```

With Other Data Types

```
int age = 20;  
String s = "Age: " + age;
```

⚠ Operator precedence matters:

```
String s = "Sum: " + 2 + 2; // Sum: 22  
String s2 = "Sum: " + (2+2); // Sum: 4
```

9. String Conversion and toString()

The `valueOf()` method converts data types into strings.

```
String s = String.valueOf(100);
```

The `toString()` method provides a string representation of an object.

Example

```
class Box {  
    double w,h,d;  
    public String toString() {  
        return w + " " + h + " " + d;  
    }  
}
```

10. Character Extraction

charAt()

```
char ch = "Java".charAt(1); // a
```

getChars()

Extracts multiple characters.

toCharArray()

Converts string to character array.

11. String Comparison

equals()

```
s1.equals(s2);
```

equalsIgnoreCase()

Ignores case differences.

12. equals() vs ==

- `equals()` → compares content
- `==` → compares reference

```
String s1 = "Hi";  
String s2 = new String("Hi");  
System.out.println(s1.equals(s2)); // true  
System.out.println(s1 == s2); // false
```

13. compareTo() and compareToIgnoreCase()

Used for dictionary-style comparison.

- Returns negative → smaller
- Zero → equal
- Positive → greater

14. Searching Strings

indexOf()

lastIndexOf()

```
String s = "Java Programming";  
s.indexOf('a');  
s.lastIndexOf('a');
```

Returns -1 if not found.

15. Modifying Strings

substring()

```
String s = "Programming";  
String sub = s.substring(3,7); // gram
```

concat()

```
String s = "Java".concat("Lang");
```

replace()

```
String s = "Hello".replace('l','w');
```

16. trim() and strip()

Removes leading and trailing whitespace.

```
String s = " Hello ".trim();
```

`strip()` removes all Unicode whitespace.

17. Data Conversion Using valueOf()

```
String s = String.valueOf(123);
```

For objects, `valueOf()` internally calls `toString()`.

18. Changing Case of Characters

```
String s = "Java";  
System.out.println(s.toUpperCase());  
System.out.println(s.toLowerCase());
```

Locale-specific versions are also available.

19. Joining Strings

The `join()` method concatenates multiple strings using a delimiter.

```
String s = String.join(" ", "Java", "is", "powerful");
```

20. Exam-Oriented Summary

- Strings are immutable
 - String class provides rich methods
 - equals() $\neq$ ==
 - Use StringBuilder for modification
 - join(), substring(), replace() are important
-

 End of Module 5 – String Handling (Full Marks Ready)